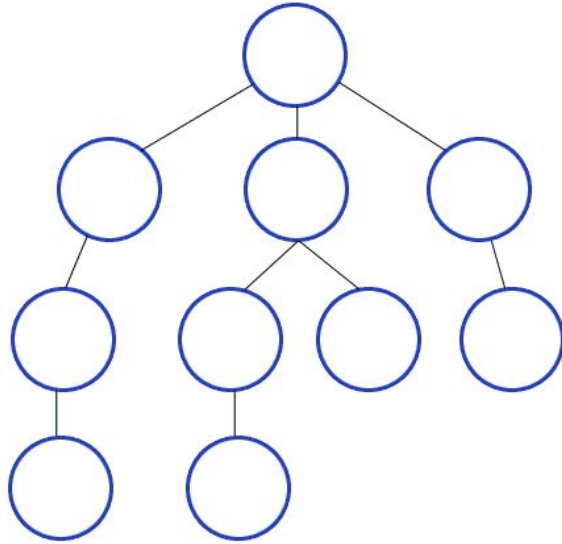


# DFS



# Lecture Flow

- 1) Pre-requisites
- 2) Problem definitions and uses
- 3) Different approaches
- 4) Checkpoint
- 5) Applications of DFS
- 6) Pair Programming
- 7) Things to pay attention (common pitfalls)
- 8) Practice questions
- 9) Resources
- 10) Quote of the day

# Prerequisites

- Introduction to Graph
- Recursion
- Stack

# Definition

- DFS is a graph **traversal algorithm**.
- It **aims to visit all nodes or vertices** of a graph in depth first manner.

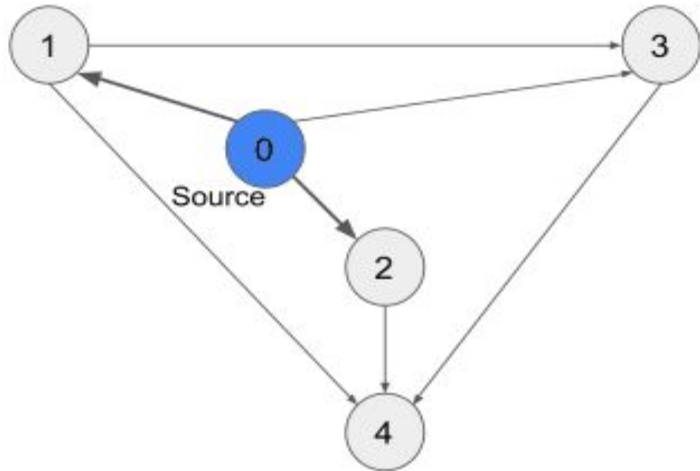
# Definition

- The algorithm starts at a particular node, known as **the source or starting node**, and **explores** as far as possible along each branch before **backtracking**.
- This means that the algorithm will visit a node and then **recursively visit all its adjacent nodes** before moving on to the next node.

# Visualization

Visited = { 0, }

source = 0, destination = 4



|   |
|---|
|   |
|   |
|   |
|   |
| 0 |

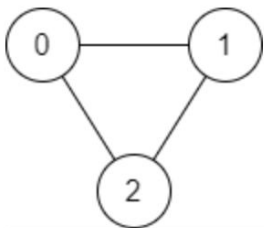
# Implementation

```
def dfs(vertex, visited):  
    # base case  
  
    visited.add(vertex)  
  
    for neighbour in graph[vertex]:  
        if neighbour not in visited:  
            dfs(neighbour, visited)
```

# Recursive and Iterative Approach of Implementing DFS

# Find if Path Exists in Graph

Example 1:



**Input:**  $n = 3$ ,  $\text{edges} = [[0,1],[1,2],[2,0]]$ ,  
 $\text{source} = 0$ ,  $\text{destination} = 2$

**Output:** true

**Explanation:** There are two paths from vertex 0  
to vertex 2:

- $0 \rightarrow 1 \rightarrow 2$
- $0 \rightarrow 2$

Easy

👍 2.8K

👎 144



Companies

There is a **bi-directional** graph with  $n$  vertices, where each vertex is labeled from  $0$  to  $n - 1$  (**inclusive**). The edges in the graph are represented as a 2D integer array `edges`, where each `edges[i] = [ui, vi]` denotes a bi-directional edge between vertex `ui` and vertex `vi`. Every vertex pair is connected by **at most one** edge, and no vertex has an edge to itself.

You want to determine if there is a **valid path** that exists from vertex `source` to vertex `destination`.

Given `edges` and the integers `n`, `source`, and `destination`, return `true` if there is a **valid path** from `source` to `destination`, or `false` otherwise.

# Recursive Approach

# Recursive Implementation

- Since it's a recursive implementation we have to obey the **3 rules of recursive functions**.
- What are those 3 rules?



# Rule 1: State

- We need to know what node we are on.
- We need to keep track of the visited nodes.

```
def dfs(node, visited):
```

## Rule 2: Base case

- For the base case, when should we stop the recursion when **the current node is our target**.
- Alternatively, we can also finish when we have **traversed over all the nodes**.

```
if node == destination:  
    return True
```

## Rule 3: Recurrence relation

- We want to **traverse all the adjacent nodes**.

```
for neighbour in graph[node]:  
    found = dfs(neighbour)  
  
    if found:  
        return True
```

```
def validPath(self, n, edges,
              source, destination):

    graph = defaultdict(list)

    for node1, node2 in edges:
        graph[node1].append(node2)
        graph[node2].append(node1)

    return dfs(source)
```

```
def dfs(node):
    if node == destination:
        return True

    for neighbour in graph[node]:
        found = dfs(neighbour)

        if found:
            return True

    return False
```

What's wrong with the above code?

# Time Complexity

- We have **V nodes** and **E edges**
- We will only **visit a node once**, and if it's **a complete graph** we will also **use every edge**.

Time Complexity =  **$O(V + E)$**

# Space Complexity

- We have **V nodes** and **E edges**
- We will store **the nodes**, and also **the edges**.

Space Complexity =  **$O(V + E)$**

Why  $O(V + E)$  and not  $O(V)$ ?

# Iterative Approach

“The Iterative implementation is just the recursive implementation done iteratively.”

- Mahatma Gandhi

As such it obeys the 3 rules as well.

# Rule 1: State

- Since we don't have access to the **call stack** we need our own stack to keep track of the current state.
- For the state, we need to **keep track of the node and the visited set.**

```
stack = [source]  
visited = set([source])
```

## Rule 2: Base case

- The base case is **similar to the recursive** implementation.
- We only need to know if the **current node is the target node**.
- Alternatively, we can also **finish** when we have **visited all the nodes**.

```
if node == destination:  
    return True
```

## Rule 3: Iteration relation

- We **visit** any **adjacent vertices** that have not yet been visited.
- For each **unvisited adjacent vertex**, we **mark it** as visited and **push it** onto the **stack**.

```
for neighbour in graph[node]:  
    if neighbour not in visited:  
        stack.append(neighbour)  
        visited.add(neighbour)
```

We will **continue to run** the loop **until** we reach the **base case** or until we **visit all the nodes**

we can visit by **starting from the source node.**

# Time Complexity

- We have **V nodes** and **E edges**
- We will only **visit a node once**, and if it's **a complete graph** we will also **use every edge**.

Time Complexity =  **$O(V + E)$**

# Space Complexity

- We have **V nodes** and **E edges**
- We will store **the nodes**, and also **the edges**.

Space Complexity =  **$O(V + E)$**

DFS on grid

# DFS on grid

- **Grid vertices** are cells, and edges connect adjacent ones.
- DFS on a grid **starts at a cell, visits its neighbors, and repeats.**
- The process continues **until all cells are visited.**

# Direction vectors

The standard square neighborhood

|            |           |            |
|------------|-----------|------------|
| $(-1, -1)$ | $(-1, 0)$ | $(-1, +1)$ |
| $(0, -1)$  |           | $(0, +1)$  |
| $(+1, -1)$ | $(+1, 0)$ | $(+1, +1)$ |

Only vertical and horizontal neighbors

|           |           |           |
|-----------|-----------|-----------|
|           | $(-1, 0)$ |           |
| $(0, -1)$ |           | $(0, +1)$ |
|           | $(+1, 0)$ |           |

The chess knight neighborhood

|            |            |  |            |            |
|------------|------------|--|------------|------------|
|            | $(-2, -1)$ |  | $(-2, +1)$ |            |
| $(-1, -2)$ |            |  |            | $(-1, +2)$ |
|            |            |  |            |            |
| $(+1, -2)$ |            |  |            | $(+1, +2)$ |
|            | $(+2, -1)$ |  | $(+2, +1)$ |            |

# Code

```
directions = [(0, 1), (0, -1), (1, 0), (-1, 0)]

visited = [[0 for i in range(len(grid[0]))] for j in range(len(grid))]

def inbound(row, col):
    return (0 <= row < len(grid) and 0 <= col < len(grid[0]))

def dfs(grid, visited, row, col):
    # base case

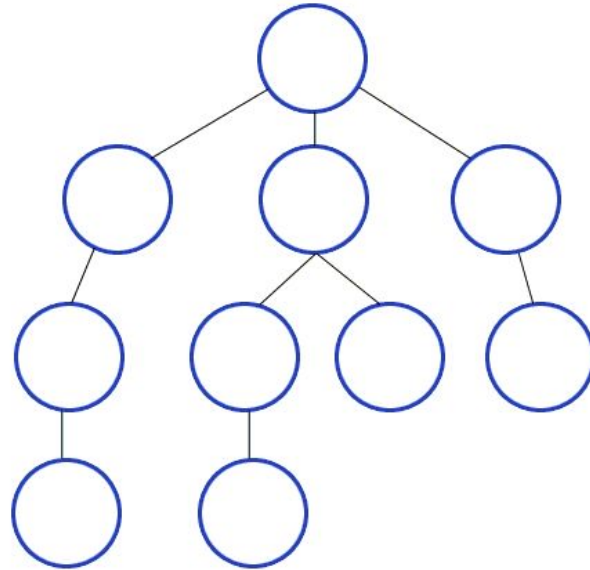
    visited[row][col] = True

    for row_change, col_change in directions:
        new_row = row + row_change
        new_col = col + col_change

        if inbound(new_row, new_col) and not visited[new_row][new_col]:
            dfs(grid, visited, new_row, new_col)
```

# Practice Problem

# DFS on Tree



# DFS on Tree

- Tree traversals (preorder, inorder, postorder) are **DFS algorithms**.
  - **Preorder** visits current, left, right
  - **inorder** visits left, current, right
  - **postorder** visits left, right, current.

# Applications

- Finding Connected Components
- Detecting Cycles
- Path Finding
- Maze Solving
- Solving Puzzles
- Generating Permutations
- Topological Sorting

# Path Finding

- DFS is a graph **traversal algorithm** that can be used for **pathfinding**.
- DFS works by **starting at a vertex** and **exploring** as far as possible along **each branch** before backtracking.

# Path Finding

- It is possible that DFS will find a **longer path before finding the shortest one.**
- While DFS can be used for pathfinding, it may **not always be the most efficient** or accurate/appropriate method.

# Sum Root To Leaf Numbers

# Time Complexity

- We have **V nodes** and **E edges**
- We will only **visit a node once**, and if it's **a complete graph** we will also **use every edge**.

Time Complexity =  $O(V + E)$

# Space Complexity

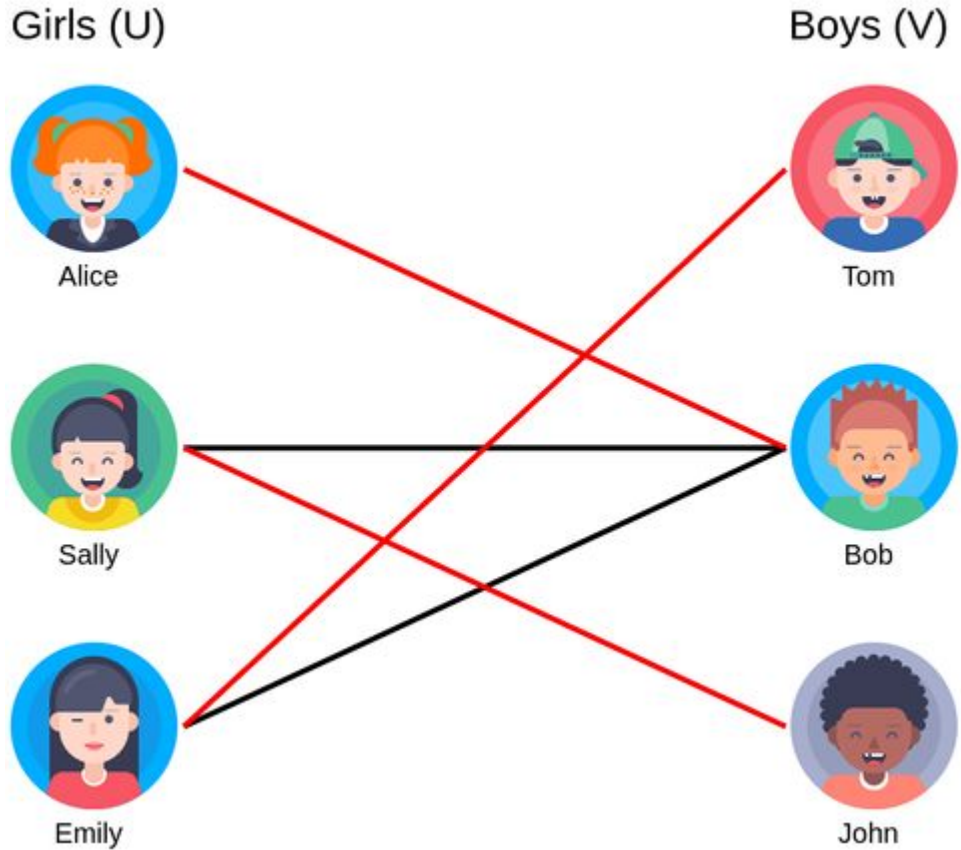
- We have **V nodes** and **E edges**
- We will store **the nodes**, and also **the edges**.

Space Complexity =  $O(V + E)$

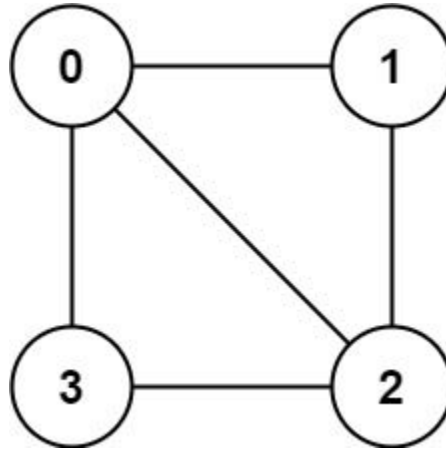
**Determine if a graph is bipartite or not?**

# Bipartite Graph

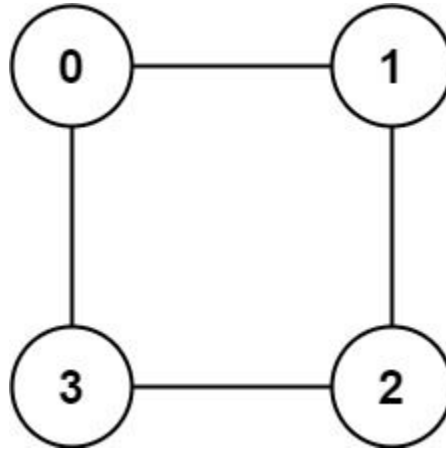
- Bipartite graphs have two sets of nodes.
- Each edge connects nodes from different sets.



Is this graph Bipartite?



Is this graph bipartite?

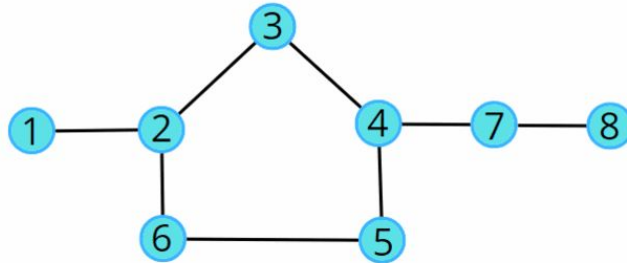


**How do we identify if a graph is bipartite or not using dfs?**

We use **DFS Coloring**.

# DFS Coloring

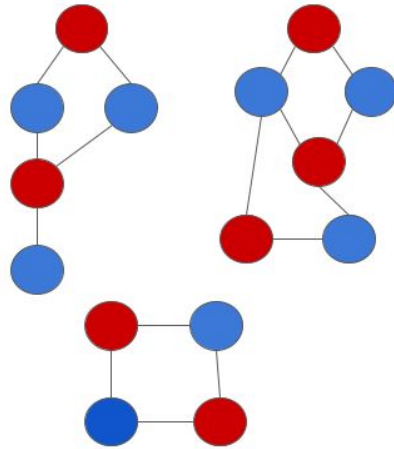
- Assign a color to **each vertex** of a graph in such a way that **no two adjacent vertices have the same color**.



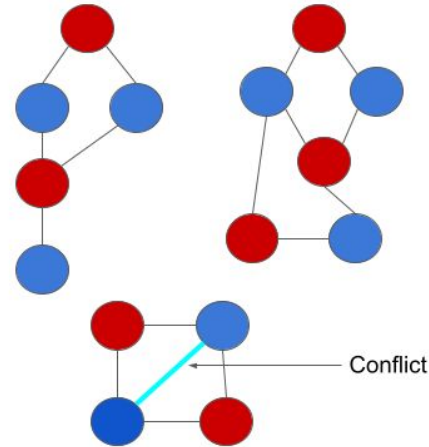
| 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  |
|----|----|----|----|----|----|----|----|
| -1 | -1 | -1 | -1 | -1 | -1 | -1 | -1 |

Colour Array

# Is a graph bipartite?



**Bipartite Graph**

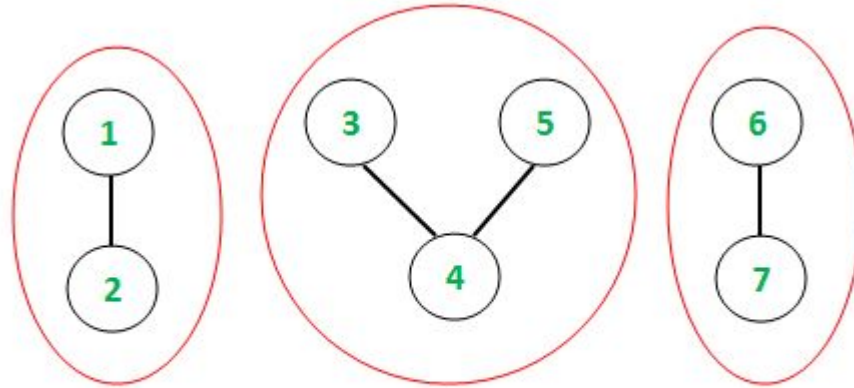


**No possible way to split**

# Practice Problem

# Connected components

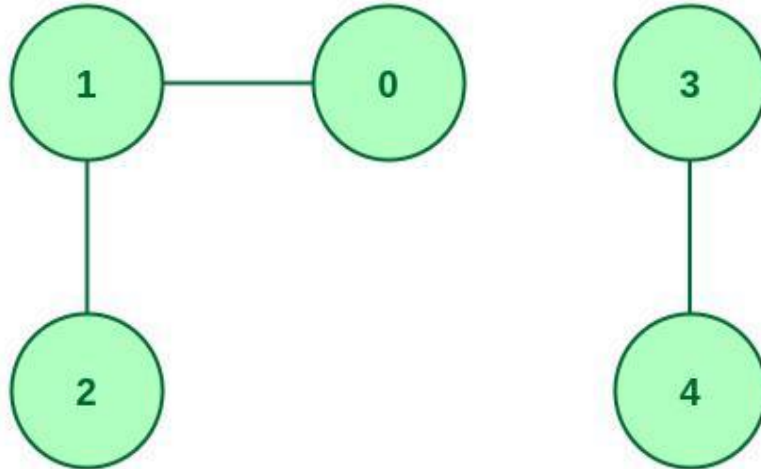
The connected parts of a graph are called its **components**



The **counts** of connected components are - 2, 3 and 2

# Finding Connected Components

A **connected component** of a graph is a **subset of vertices in the graph** such that there is a **path between any two vertices** in the subset.



Brainstorm on how to find connected components.



# Number of Islands

## 200. Number of Islands

Medium

👍 19339

💬 428

♡ Add to List

🔗 Share

Given an  $m \times n$  2D binary grid `grid` which represents a map of '1' s (land) and '0' s (water), return *the number of islands*.

An **island** is surrounded by water and is formed by connecting adjacent lands horizontally or vertically. You may assume all four edges of the grid are all surrounded by water.

Example 1:

```
Input: grid = [
  ["1","1","1","1","0"],
  ["1","1","0","1","0"],
  ["1","1","0","0","0"],
  ["0","0","0","0","0"]
]
```

Output: 1

Here's how we can use DFS:

1. **Initialize** all **vertices** as unvisited.

- 
2. For each **unvisited vertex**, perform a **DFS starting** from that vertex

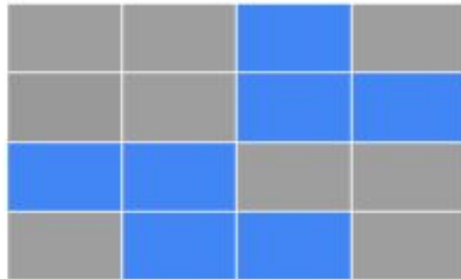
3. Mark all **visited vertices** as part of the **same connected component** as the **starting vertex**.

4. **Repeat** steps 2-3 for any **remaining unvisited vertices** until **all vertices** have **been visited**.

After this process, **the set of marked vertices** for **each DFS traversal** will give you the **connected components** of the graph.

# Visualization

Visited = { }



count = 0

Call stack

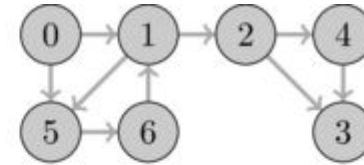


How can we detect cycles in directed graph using dfs?

# Cycle detection

- We will run a series of DFS in the graph.
- Initially all vertices are colored **white (0)**
- From each unvisited (**white**) vertex, start the DFS, mark it **gray (1)** while entering and mark it **black (2)** on exit
- If DFS moves to a **gray** vertex, then we have found a cycle

# Visualization



- open node
- closed node
- unvisited node
- current node
- tree edge
- -> cross edge
- -> cycle edge
- -> forward edge
- unvisited edge

starting dfs

Practice problem

# Common Pitfalls

**Infinite loops:** DFS can get stuck in an infinite loop if it encounters a cycle in the graph. To avoid this, it is important to keep track of visited nodes and avoid revisiting them.

# Common Pitfalls

## Stack overflow & Exceeding Maximum Recursion Depth

- DFS uses a stack to keep track of nodes to visit. If the graph is very deep or has a large number of branches, the stack can become very large and cause a stack overflow.
- If you are using Python you are aware that the maximum recursion depth is around 1000, in some cases we might have more than 1000 nodes in our call stack in these cases we might be faced by **maximum recursion depth exceeded error**

# Common Pitfalls

- To fix the maximum recursion depth exceeded error we can manually increase the recursion limit.
- To fix the stack overflow error we can manually increase the stack size for our python program.  
Taken together it will look like the image on the right.

```
import sys, threading

def main():
    # your code goes here

sys.setrecursionlimit(1 << 30)
threading.stack_size(1 << 27)

main_thread = threading.Thread(target=main)
main_thread.start()
main_thread.join()
```

# Common Pitfalls

**Choosing the wrong starting node:** The output of DFS can depend on the starting node. If the starting node is not chosen carefully, it may not be possible to reach some nodes in the graph. It is important to consider the structure of the graph and the problem at hand when choosing the starting node.

Recap

# Recap Points

- DFS Definition and Algorithm
- DFS Applications
- Visual: summary of DFS algorithm on a graph

# Resources

[GeeksForGeeks](#)

[Leetcode solution](#)

Chat GPT

# Practice Problems

[Employee-importance](#) ✓

[Number-of-provinces](#) ✓

[Sum-of-nodes-with-even](#) [-valued-grandparent](#) ✓

[Max-area-of-island](#) ✓

[Evaluate-division](#) ✓

[Sum-root-to-leaf-numbers](#) ✓

[Detonate-the-maximum-bombs](#) ✓

[Surrounded-regions](#) ✓

[Minesweeper](#) ✓

[Lowest-common-ancestor-of-deepest-leaves](#) ✓

[recover-binary-search-tree](#)

[Check if there is a valid path in a grid](#)

You have to write your own story, your own graph and rhythm.